

Map2Check

Modos de verificação, propriedades, sub-propriedades
e cobertura de CWEs

Levantamento a partir do código-fonte
branch `wasm-verification-upgrade` — Map2Check v8.0.0

6 de setembro de 2026

Resumo

Este documento inventaria *tudo* que o Map2Check consegue verificar hoje: os seis modos de verificação implementados, as especificações SV-COMP que cada um responde, os dez veredictos que o runtime é capaz de escrever, e o mapeamento CWE→modo tal como ele existe nos dois harnesses de avaliação do repositório (CASTLE e Juliet). Nada aqui é aspiracional: cada entrada cita o arquivo e a linha que a sustenta, e as limitações conhecidas estão na Seção 7 em vez de omitidas.

Sumário

1	Escopo e fontes	1
2	Modos de verificação	2
2.1	Especificação SV-COMP correspondente	2
2.2	Pipeline de passes por modo	3
3	Veredictos e sub-propriedades	3
4	Operações instrumentadas em <code>OVERFLOW_MODE</code>	5
5	Cobertura de CWEs	5
5.1	Mapeamento consolidado	5
5.2	CWEs explicitamente fora de escopo	6
5.3	Cobertura no modo WebAssembly	6
6	Eixos ortogonais	7
7	Limitações conhecidas	7
8	Resumo quantitativo	9

1 Escopo e fontes

O levantamento foi feito sobre quatro camadas do código, porque a resposta completa não está em nenhuma delas isoladamente:

1. **CLI** — `modules/frontend/map2check.cpp` define as flags e traduz cada uma em um modo.

2. **Orquestração** — `modules/frontend/caller.hpp:21` declara `enum class Map2CheckMode`; `caller.cpp:389` decide quais *passes* LLVM cada modo carrega.
3. **Instrumentação** — `modules/backend/pass/*.cpp` são os passes que injetam as chamadas de verificação no IR.
4. **Runtime** — `modules/backend/library/lib/AnalysisMode*.c` implementam a checagem em tempo de execução e escrevem o veredicto; `header/Map2CheckTypes.h:68` enumera os veredictos possíveis.

A cobertura de CWEs vem dos dois *runners* de avaliação versionados no repositório, que são a única declaração executável desse mapeamento: `tests/castle/run_castle_evaluation.sh` e `tests/juliet/run_juliet_evaluation.sh`. O contrato entre eles é verificado por `tests/integration/test_cwe_mode_mapping.sh`, que impede que qualquer CWE do benchmark caia num modo não declarado.

2 Modos de verificação

São seis modos, dos quais cinco checam uma propriedade e um — Cover-Branches — não checa propriedade alguma, existindo apenas para produzir suítes de teste.

#	Modo	Flag	O que checa
1	MEMTRACK_MODE (padrão)	--memtrack	Segurança de memória: <i>free</i> inválido, referência inválida e vazamento de memória alocada.
2	MEMCLEANUP_MODE	--memcleanup-property	Toda memória alocada foi liberada até o fim do <code>main</code> .
3	OVERFLOW_MODE	--check-overflow	Overflow de inteiro <i>com sinal</i> e divisão/resto por zero.
4	REACHABILITY_MODE	--target-function --target-function-name	Se uma função-alvo nomeada é alcançável.
5	ASSERT_MODE	--check-asserts	Se algum <code>__VERIFIER_assert</code> ou <code>__assert_fail</code> pode falhar.
6	COVER_BRANCHES_MODE	--cover-branches	<i>Nada</i> . Explora caminhos e registra suas entradas (Test-Comp Cover-Branches).

Tabela 1: Os seis modos. Fonte: `caller.hpp:21-37` e `map2check.cpp:781-878`.

2.1 Especificação SV-COMP correspondente

É preciso separar duas coisas que o código trata em lugares diferentes, e que não coincidem.

Na *witness*. A especificação é escolhida pela **propriedade violada**, não pelo modo — `witness/graph.cpp:208-237` faz um `switch` sobre `PropertyViolated`.

Veredicto	Especificação emitida
FALSE-FREE	CHECK(init(main()), LTL(G valid-free))
FALSE-DEREF	CHECK(init(main()), LTL(G valid-deref))
FALSE-MEMTRACK	CHECK(init(main()), LTL(G valid-memtrack))
FALSE-OVERFLOW	CHECK(init(main()), LTL(G ! overflow))
TARGET-REACHED	especificação de alvo (SpecificationType::TARGET)
NONE (<i>TRUE</i>)	conforme spectTrue: alvo, ! overflow, ou — por padrão — as três linhas de valid-free + valid-deref + valid-memtrack (MEMSAFETY).

Tabela 2: Fonte: witness/witness.hpp:64-101 (enum SpecificationType) e witness/graph.cpp:198-258. **Não existe especificação de memcleanup:** o enum SpecificationType tem seis valores e nenhum deles é valid-memcleanup. Ver a Seção 7, item 3.

Nos metadados do *test-suite*. Aqui a especificação vem do arquivo passado em `--property-file`, copiado *verbatim*. As duas strings abaixo são apenas o *fallback* para quando esse arquivo não é passado ou não pode ser lido (map2check.cpp:104-128):

Modo	<i>Fallback</i> de <specification>
REACHABILITY	COVER(init(main()), FQL(COVER EDGES(@CALL(reach_error))))
demais modos	COVER(init(main()), FQL(COVER EDGES(@DECISIONEDGE)))

2.2 Pipeline de passes por modo

Cada modo monta uma sequência distinta de *passes* do New Pass Manager. O `nondet-pass` e o `map2check-library` são carregados sempre.

Modo	Pass adicional	Runtime
MEMTRACK	MemoryTrackPass	AnalysisModeMemtrack.c
MEMCLEANUP	MemoryTrackPass	AnalysisModeMemcleanup.c
OVERFLOW	OverflowPass	AnalysisModeOverflow.c
REACHABILITY	TargetPass	AnalysisModeNone.c
ASSERT	AssertPass	AnalysisModeAssert.c
COVER_BRANCHES	<i>nenhum</i>	AnalysisModeNone.c

Tabela 3: Passes: caller.cpp:389-445; *runtime* linkado: caller.cpp:499-531. REACHABILITY não precisa de um AnalysisMode próprio porque TARGET-REACHED é escrito pela biblioteca comum (Map2CheckFunctions.c:62). Cover-Branched é deliberadamente o pipeline mais enxuto: men6eetaTm (cão)sign27(eciemais)-330(maio)-3minh (men6e)explorarregadospipoupa7(eradamenos333(3.) TJ

Cód.	Veredicto	Emitido por	Significado
0	NONE	todos	Nenhuma violação encontrada (<i>TRUE</i>).
1	UNKNOWN (nunca gravado)	—	Indeciso. As duas chamadas que o escreveriam estão comentadas (<code>AnalysisModeMemtrack.c:308</code> , <code>AnalysisModeMemcleanup.c:298</code>); ver a Seção 7, item 11.
2	FALSE-FREE	Memtrack, Memcleanup	<i>free</i> de ponteiro inválido, <i>double free</i> , <i>free</i> de memória não alocada. Também o <i>free</i> de WASM sobre <i>offset</i> não alocado.
3	FALSE-DEREF	Memtrack	Dereferência de ponteiro inválido (nulo, liberado, fora da alocação).
4	FALSE-MEMTRACK	Memtrack	Memória alocada e não liberada (<i>leak</i>).
5	TARGET-REACHED	TargetPass	A função-alvo é alcançável.
6	FALSE-OVERFLOW (gravado como OVERFLOW)	Overflow; Memtrack (WASM)	Overflow em aritmética de inteiro com sinal. No modo WASM é também o veredicto de acesso fora dos limites da <i>linear memory</i> (<code>AnalysisModeMemtrack.c:418-424</code>).
7	ASSERT	AssertPass	Assertiva do usuário violada.
8	FALSE-MEMCLEANUP	Memcleanup	Memória ainda viva no retorno do <code>main</code> .
9	FALSE-DIVBYZERO	Overflow	Divisão ou resto por zero.

Tabela 4: Fonte: `library/header/Map2CheckTypes.h:68-79` (enum `ViolatedProperty`) e `frontend/utils/tools.hpp:105-116` (`PropertyViolated`). Os pontos de emissão estão em `AnalysisModeMemtrack.c:141,174,299,356,413,421`, `AnalysisModeMemcleanup.c:142,170,346`, `AnalysisModeOverflow.c:193` e `AnalysisModeAssert.c:20`.

Observação importante. `FALSE-DIVBYZERO` *não tem flag própria*: divisão por zero é detectada como efeito colateral de `--check-overflow`, porque a instrumentação de `SDiv/SRem` é a mesma que checa o overflow de `INT_MIN / -1`. Qualquer comparação com outra ferramenta precisa acionar `--check-overflow` para cobrir CWE-369.

4 Operações instrumentadas em OVERFLOW_MODE

Instrução LLVM	Instrumentada	Nota
Add, Sub, Mul	sim	Apenas a variante <i>com sinal</i> . O critério é a <i>flag nsw</i> da instrução: <code>isUnitAssignment = !binOp->hasNoSignedWrap()</code> , então só a aritmética que o clang marcou como <i>nsw</i> é instrumentada. As versões <code>uint</code> existem no código porém estão comentadas.
SDiv	sim	Gera FALSE-OVERFLOW (<code>INT_MIN / -1</code>) ou FALSE-DIVBYZERO.
SRem	sim	Idem.
UDiv, URem	não	Aritmética sem sinal não faz parte da propriedade <i>no-overflow</i> .
Shl, LShr, AShr	não	O case de <code>Shl</code> existe no <code>switch</code> mas tem corpo vazio — deslocamento à esquerda com perda de bits <i>não</i> é detectado.
And, Or, Xor	não	<i>Case</i> vazio.
FAdd, FSub, FMul, FDiv, FRem	não	<i>No-op</i> deliberado: o SV-COMP define <i>no-overflow</i> apenas sobre tipos inteiros.

Tabela 5: Fonte: `pass/OverflowPass.cpp:325-424`. São **cinco** as instruções efetivamente instrumentadas. A propriedade é avaliada sobre o *tipo resultante da operação*, conforme a definição do SV-COMP — ver o comentário em `OverflowPass.cpp:418` e `pass/OperationsFunctions.hpp:63`.

5 Cobertura de CWEs

O repositório contém dois mapeamentos `CWE→modo`, ambos executáveis e ambos versionados. Eles não são idênticos porque os benchmarks não são: o CASTLE-C250 cobre CWEs de segurança mais amplas, e o Juliet cobre famílias de memória e aritmética com muitas variantes por família.

5.1 Mapeamento consolidado

Tabela 6: União dos dois mapeamentos. **15 CWEs** distintos são verificáveis. Fonte: `tests/castle/run_castle_evaluation.sh (CWE_MODE)` e `tests/juliet/run_juliet_evaluation.sh:108-126`. “—” significa que o CWE não está naquele benchmark, não que seja insuportado.

CWE	Descrição	Modo	CASTLE	Juliet
<i>Segurança de memória — -memtrack</i>				
121	Stack-based buffer overflow	--memtrack	—	sim
122	Heap-based buffer overflow	--memtrack	—	sim
125	Out-of-bounds read	--memtrack	sim	—
415	Double free	--memtrack	sim	sim
416	Use after free	--memtrack	sim	sim
476	NULL pointer dereference	--memtrack	sim	sim
761	Free of pointer not at start of buffer	--memtrack	sim	sim
787	Out-of-bounds write	--memtrack	sim	—

continua na próxima página

(continuação da Tabela 6)

CWE	Descrição	Modo	CASTLE	Juliet
822	Untrusted pointer dereference	--memtrack	sim	—
843	Type confusion (<i>access of resource with incompatible type</i>)	--memtrack	sim	—
<i>Aritmética — --check-overflow</i>				
190	Integer overflow / wraparound	--check-overflow	sim	sim
191	Integer underflow	--check-overflow	—	sim
369	Divide by zero	--check-overflow	sim	sim
<i>Ciclo de vida de memória — --memcleanup-property</i>				
401	Missing release of memory (<i>memory leak</i>)	--memcleanup-property	sim	sim
<i>Assertivas — --check-asserts</i>				
617	Reachable assertion	--check-asserts	sim	—

5.2 CWEs explicitamente fora de escopo

O runner do CASTLE declara treze CWEs como fora de escopo. Isso é uma decisão deliberada e testada, não uma omissão: `test_cwe_mode_mapping.sh` exige que todo CWE do benchmark esteja ou mapeado ou declarado fora de escopo, justamente para que nada caia num modo degenerado em silêncio.

CWEs	Motivo
628, 674, 770, 835	Terminação e consumo de recurso (recursão descontrolada, alocação sem limite, laço infinito). O Map2Check não implementa a propriedade de terminação.
22, 78, 89, 134	Injeção e uso indevido de dados externos (<i>path traversal</i> , injeção de comando/SQL, <i>format string</i>). Exigem modelo de <i>taint</i> , ausente.
253, 327, 362, 522, 798	Verificação incorreta de retorno, criptografia fraca, condição de corrida, credenciais desprotegidas/embutidas. Nenhuma é expressável como uma das cinco propriedades implementadas.

Tabela 7: Fonte: `run_castle_evaluation.sh:49` (OUT_OF_SCOPE_CWES).

5.3 Cobertura no modo WebAssembly

Com `--wasm` a entrada é um binário `.wasm` elevado a C via `wasm2c` (WABT $\geq 1.0.41$) e daí a LLVM-IR. A análise é *agnóstica à linguagem-fonte*: instrumenta a *linear memory* e as chamadas `wasm_rt_*`, comuns a todo módulo elevado. O modo é forçado para `MEMTRACK_MODE`.

Propriedade	CWEs de exemplo	Estado
Acesso fora dos limites da <i>linear memory</i> (reportado como FALSE-OVERFLOW, não FALSE-DEREF)	121, 122, 124, 126, 127	sim
Overflow aritmético	190	sim
Alcançabilidade de função-alvo/erro	—	sim
<i>Leak</i> / <i>use-after-free</i> / <i>double free</i>	401, 416, 415	não

Tabela 8: Fonte: README.md:92–99. A última linha é trabalho futuro por uma razão estrutural: no código elevado, o alocador da linguagem-fonte é compilado *dentro* da linear memory, então as chamadas `malloc/free` não existem no IR. Rastrear memória ali exigiria uma convenção de exportação ou heurística de detecção de alocador.

6 Eixos ortogonais

Não são propriedades, mas multiplicam a matriz de execução e precisam ser fixados em qualquer comparação para que ela seja justa.

Eixo	Valores
Gerador de <i>nondet</i> Solucionador SMT	fuzzer (LibFuzzer), symex (KLEE), nenhum z3 (padrão) e stp passam direto ao KLEE; btor e yices2 são roteados por metasmt e não funcionam na imagem de referência (ver Seção 7, item 9)
Entrada	C / .i / .bc; ou .wasm via --wasm
Função de entrada	--entry-function (padrão main)
Modificadores	--slice, --seed-exchange, --add-invariants, --btree
Saídas	--generate-witness, --generate-test-suite, --generate-testcase, --print-counter

Tabela 9: Fonte: map2check.cpp:719–766.

Funções *nondet* reconhecidas (16). `__VERIFIER_nondet_` seguido de: `int`, `uint`, `unsigned`, `char`, `uchar`, `pchar`, `short`, `ushort`, `long`, `ulong`, `bool`, `pointer`, `size_t`, `loff_t`, `sector_t`, `double` (pass/NonDetPass.cpp:155–171). Além disso, `__VERIFIER_assume` e `assume_abort_if_not` são reescritos como poda de caminho em vez de `abort()` (NonDetPass.cpp:33–99) — sem isso, uma suposição falhando deixava um `abort.err` indistinguível de uma violação real.

Códigos de saída. 0 sucesso; 1 erro de linha de comando; 3 a opção é válida mas esta compilação não tem a capacidade (é o caso de `--add-invariants` sem `-DENABLE_CLAM=ON`); 4 o veredicto divergiu de `--expected-result`. O código 4 substituiu um `abort()`, que fazia o BenchExec ler “resposta errada” como “ferramenta quebrada” (map2check.cpp:47–60).

7 Limitações conhecidas

Registradas aqui porque uma comparação que as ignore mede a coisa errada.

1. **Overflow sem sinal não é instrumentado.** As variantes `uint` de `Add/Sub/Mul` estão no código, porém comentadas. Isso está alinhado com a definição do SV-COMP, mas diverge do que uma ferramenta genérica de análise poderia reportar.
2. **Ponto flutuante é *no-op* deliberado.** `FAdd/FSub/FMul/FDiv/FRem` não geram veredicto.

3. **Deslocamento à esquerda não é checado.** O case de Shl em OverflowPass.cpp:395 tem corpo vazio, como LShr, AShr, And, Or e Xor. Um $x \ll n$ que perde bits significativos passa como *TRUE*.
4. **Divisão por zero não tem flag própria** — só é alcançada via `--check-overflow`.
5. **Três veredictos geram uma *witness* de corretude apesar de serem violações.** O switch de witness/graph.cpp:208-237 tem case para FALSE-FREE, FALSE-DEREF, FALSE-MEMTRACK, TARGET-REACHED e FALSE-OVERFLOW. ASSERT, FALSE-MEMCLEANUP e FALSE-DIVBYZERO caem no default, que constrói um CorrectnessWitnessGraph — e, sem spectTrue, o rotula com a especificação MEMSAFETY. Afeta apenas execuções com `--generate-witness`; o veredicto impresso e o código de saída continuam corretos.
6. **Alcançabilidade com alvo main é degenerada.** O TargetPass instrumenta *sítios de chamada* cuja função chamada tem o nome dado; um programa nunca chama o próprio main. Na baseline v6, 98 de 217 execuções passaram por esse modo e produziram zero FALSE em 59 programas sabidamente vulneráveis. O teste test_cwe_mode_mapping.sh hoje proíbe esse mapeamento.
7. **Invariantes (Clam) são *opt-in*.** Um invariante não-sólido produz um TRUE errado em vez de um erro, o que é a falha mais cara possível — por isso exige `-DENABLE_CLAM=ON` e o driver instalado.
8. ***Slicing*, *seed-exchange* e WASM rodam nos defaults** da biblioteca; estão integrados, não ajustados. Qualquer número medido sobre eles reflete a configuração padrão, não um limite da técnica.
9. **Dois dos quatro solucionadores não existem no build.** A CLI aceita z3, stp, btor e yices2 (map2check.cpp:804), mas btor e yices2 são entregues ao KLEE como `--solver-backend=metasmt` (caller.cpp:748-758) e o Dockerfile.dev compila o KLEE apenas com ENABLE_SOLVER_Z3 e ENABLE_SOLVER_STP — sem metaSMT, sem Boolector, sem Yices. Passar um deles é aceito na linha de comando e falha na execução.
10. **O veredicto de overflow é gravado como OVERFLOW, não FALSE-OVERFLOW.** PropertyGenerator.c:97 escreve a string OVERFLOW em map2check_property, e é isso que tools.cpp:228 lê de volta. FALSE-OVERFLOW é o nome do valor do *enum*, não o texto no arquivo: um harness que procure FALSE-OVERFLOW não casa com nada.
11. **UNKNOWN nunca é gravado, e o arquivo alternativo tem o nome errado.** O switch de PropertyGenerator.c tem case UNKNOWN:, mas as duas únicas chamadas write_property(UNKNOWN, ...) estão comentadas (AnalysisModeMemtrack.c:308, AnalysisModeMemcleanup.c:298) — é código morto, do mesmo tipo do case de Shl. O runtime usa write_property_unknown(), que escreve num arquivo à parte chamado map2check_property__unknown (dois sublinhados, PropertyGenerator.c:16); o frontend abre map2check_property_unknown (um sublinhado, tools.cpp:114). Os dois nomes não coincidem, então esse arquivo também nunca é lido. O tool ainda *reporta* VERIFICATION UNKNOWN, mas por outras rotas — o inicializador do campo (tools.hpp:136), o map2check_property ausente ou ilegível (tools.cpp:102,252) e o próprio frontend em caso de estouro de tempo (map2check.cpp:581,585). O que não existe é o caminho pelo qual o *runtime* declararia indecisão.
12. **KLEE: --max-time exige segundos inteiros.** Um valor fracionário encerra a execução em silêncio.

8 Resumo quantitativo

Dimensão	Total
Modos de verificação implementados	6
dos quais checam uma propriedade	5
Valores do <i>enum</i> <code>ViolatedProperty</code>	10
realmente gravados em <code>map2check_property</code>	9
que são violações concretas	8
que geram <i>witness</i> de violação	5
Tipos de especificação de <i>witness</i> (<code>SpecificationType</code>)	6
Instruções LLVM instrumentadas para overflow	5
Funções <i>nondet</i> reconhecidas	16
CWEs verificáveis (união CASTLE $\cup$ Juliet)	15
CWEs declarados fora de escopo	13
Solucionadores SMT aceitos pela CLI	4
utilizáveis na imagem de referência (Z3, STP)	2
Geradores de entrada selecionáveis (<code>--nondet-generator</code>)	2